

PROJECT REPORT

AES Vault

A Password-Based AES-256-GCM Secure File Encryption and Decryption System

Covering: Project Proposal · Literature Review · System Design · Testing & Evaluation · Implementation

Maralbyek Tilyek

Abstract

AES Vault is a file encryption and decryption system that uses a password as its key, and is designed to serve as an example of the practical use of modern authenticated cryptography. Arbitrary file data such as text files, documents, images, compressed files, binary files, and empty files are protected with AES-256-GCM. AES-GCM offers confidentiality and integrity: When the file is encrypted, it cannot be accepted if the encryption is wrong, or if the password is wrong, or if the nonce or authentication tag are wrong.

Users input a password and the system generates a 256-bit AES key using PBKDF2-HMAC-SHA256. The user enters a password and a 256-bit AES key is generated from that password via PBKDF2-HMAC-SHA256. Each file encrypted with a new cryptographically secure 16-byte salt, 480,000 key-derivation iterations to make the password-guessing more expensive. A new 12-byte nonce is also used for each encryption operation. The encrypted file is composed of a versioned header, iteration count, salt, nonce, authentication tag and ciphertext. The password and derived key will not be saved in the encrypted container.

Implementing AES Vault in Python using libraries: AES-GCM and PBKDF2-HMAC-SHA256. A light-weight Python HTTP server is used to link the cryptographic engine with HTML, CSS and JavaScript interface. It uses temporary directories for uploaded files, won't overwrite existing destinations accidentally, and does atomic writes to ensure that unfinished operations don't overwrite good files. Non-sensitive operation information like operation type, size, file name and time stamp are stored in a local SQLite database.

Testing confirmed true round-trip recovery, binary data handling, rejected incorrect passwords, rejected tampered ciphers, rejected malformed files, and existing-destination protection. The automated tests passed all five tests. The test machine achieved 1.47 seconds in encryption and 0.87 seconds in decryption with a 100 MB benchmark and the recovered file was identical to the original data. The results indicate that the project achieves the primary objectives of confidentiality, tamper detection, password based key derivation, safe file handling and usability.

AES Vault is a prototype, still in the local, single-user education space. Its security relies on the strength of the selected password and it offers no password recovery, multi-user access control, cloud storage, hardware security modules or protection if the attacker already has control of an unlocked session.

AES-256-GCM, PBKDF2, Authenticated Encryption, File Security, Cryptography in Python, Password-Based Encryption.

Table of Contents

1.0 Project Proposal	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Aim	5
1.4 Objectives	6
1.5 Project Scope	6
1.5.1 In Scope	6
1.5.2 Out of Scope	6
1.6 Proposed Methodology	6
1.6.1 Encryption Workflow	6
1.6.2 Decryption Workflow	7
1.6.3 Threat Model and Security Considerations	7
1.7 Tools and Technologies (as proposed)	7
1.8 Expected Outcome	8
1.9 Testing and Evaluation Criteria (as proposed)	8
2.0 Literature Review	9
2.1 Introduction	9
2.2 Symmetric-Key Cryptography	9
2.3 The Advanced Encryption Standard (AES)	9
2.4 AES-128 and AES-256	10
2.5 Authenticated Encryption and AES-GCM	10
2.6 Passwords, Keys, and PBKDF2	10
2.7 Salt, Nonce, Authentication Tag, and File Format	10
2.8 Secure Software Implementation	11
2.9 Related Applications and Security Considerations	11
2.10 Relevance to the Implemented Project	11
3.0 System Design	12
3.1 Introduction	12
3.2 System Overview	12
3.3 System Architecture	12
3.4 System Design Diagrams	13
3.4.1 Use Case Diagram	13
3.4.2 Data Flow Diagram	14
3.4.3 Application Workflow Diagram	15
3.5 Encryption and Decryption Process Summary	16
3.6 Cryptographic Design	16
3.6.1 Password and Key Derivation	16
3.6.2 AES-256-GCM	16
3.6.3 Salt and Nonce Generation	16
3.6.4 Authentication Tag	17
3.7 User Interface Design	17

3.8 Code Flow-----	17
3.9 Version Control-----	18
4.0 Testing and Evaluation-----	19
4.1 Testing Approach-----	19
4.2 Automated Test Suite-----	19
4.3 Performance Evaluation-----	19
4.4 Live Application Evidence-----	19
4.5 Operation History (Database Evidence)-----	21
4.6 Evaluation Summary-----	22
5.0 Implementation-----	22
5.1 From Design to Implementation: An Honest Note on Architecture-----	23
5.2 Delivered Architecture-----	23
5.2 Tools and Technologies used in implementation phase-----	24
5.3 Cryptographic Engine Implementation-----	25
5.3.1 Key Implementation Details-----	25
5.4 Encrypted File Format (as implemented)-----	25
5.5 Web Server and API Implementation-----	26
5.6 Source Code Evidence-----	27
5.7 Deployment Configuration-----	27
5.8 How to Run the Project-----	28
5.9 Language Distribution-----	28
References-----	29
Appendix A — Project Repository and Deliverables-----	29

PHASE 1

Project Proposal

Rubric weight: 5%

This phase defines the problem the system addresses, the cryptographic technique selected, the intended security goals, and the boundaries of the project, as approved at the start of the semester.

1.1 Background

Digital files can contain information that is of value to people other than the owner: academic records, financial statements, medical records, source code, personal identification documents, and more. Files are not secured by an encrypted laptop, removable drive or shared network location, so any individual who has access to the storage media can read, copy, and modify the files as they choose, both physically and remotely. Encryption is used to address the problem of exposure by converting plaintext to ciphertext, making it difficult to decrypt without the secret key.

The Advanced Encryption Standard (AES) is the result of a multi-year, public process of evaluating candidate ciphers overseen by the U.S. National Institute of Standards and Technology (NIST) that was recommended for adoption in FIPS PUB 197 (2001). It has been the de facto standard for data protection at rest since that time and is the basis of technologies like full-disk encryption systems (BitLocker, FileVault) and secure messaging and TLS. The project is aimed at bridging this gap between the theory of cryptography and a usable application, by developing a end user AES application that is properly implemented and can be used by a non-technical user, with the authenticated Galois/Counter Mode (AES-GCM) variant of AES.

1.2 Problem Statement

Most computer users don't use good, standard encryption on sensitive files - by hiding a file in a hidden folder or by using consumer-grade archive software with weak or old ciphers as the default. None of them offer true confidentiality from an opponent that can physically access the storage device, and few offer any way to determine if the file has been modified.

Many applications and code samples on the web, on the contrary, implement AES incorrectly: they use Electronic Codebook (ECB) mode, which allows an attacker to identify patterns in the clear text; they create a cryptographic key from the password itself (offline brute force attacks becomes easy); or they don't include any integrity/authentication mechanism (easy for an attacker to modify the cipher text without being noticed).

1.3 Aim

To develop an application that is user friendly and easy to use for password based encryption and decryption of files using AES-256 (AES-GCM) with a password based key: in one convenient package, confidentiality, tamper detection and password based authentication.

1.4 Objectives

1. To understand and discuss the fundamentals of AES, symmetric and password based key derivation (PBKDF2), AEAD and why all of them are chosen.
2. To create a Use Case Diagram, Data Flow Diagram and Flowchart for encryption and decryption.
3. To apply a unique, cryptographically random salt per file and a number of iterations as recommended by current NIST/OWASP guidelines to create a secure password.
4. To Encrypt a file with AES 256-GCM and use random nonces that are different for each operation; before writing any decrypted data to disk, check the authenticity tag.
5. To design a GUI and successfully implement a password entry system allowing the user to select a file and enter a masked password, enter a confirmed password, display a progress bar when a large file is selected, and give clear and non-technical success/failure messages.
6. To systematically test the tool using various file types, file sizes, correct and incorrect passwords and intentional corrupted cipher text, to verify that the tool is functioning properly and that it is rejecting cases of incorrect decryption attempts.
7. To produce the final report to record the design of the system, the decisions made during its implementation, the analysis of threats it might face, the testing of the system and the limitations and future additions.

1.5 Project Scope

1.5.1 In Scope

This utility is used to encrypt and decrypt files in local mode, per user. It is capable of handling standard file formats (text, PDF, images, compressed files), encrypting the raw bytes of the file, not just the text. There is no key file that the user has to store to decrypt all of the output files, and each output file contains its own nonce, ciphertext, random salt, and Ciphertext Authentication Tag (CAT).

1.5.2 Out of Scope

1. Storing files in the cloud or transferring files via a network.
2. Multiple user account management and/or access control lists or role based permissions.
3. Password recovery mechanisms — by design a forgotten password should make the file...
4. Not impossible to recover from: This is an essential security property not a restriction.
5. Enterprise level key management (key management modules or hardware security modules [HSM]).
6. Attack resistance against side-channel attacks, cold-boot attacks (which require access to memory of the running system).

1.6 Proposed Methodology

1.6.1 Encryption Workflow

1. The user uses a file-picker to select a file, and enters an encrypted password in a text box, which is then checked to see if it is misspelled in another text box.
2. A secure random number generator (Python secrets / os.urandom) is used to generate a cryptographically random 16-byte salt.
3. The current OWASP guidance on password storage, which was adapted for key derivation, recommends 256-bit AES key that is 100% generated with PBKDF2-HMAC-SHA256 using a password and a salt, with a minimum of 480,000 iterations.
4. The contents are read and then encrypted with AES-256-GCM, using a new nonce, which produces a 16-byte authentication tag and ciphertext.

5. The original file is not modified and output to a new file (e.g., document.pdf.enc) in some fixed binary header format: salt, nonce, authentication tag and ciphertext.

1.6.2 Decryption Workflow

1. The user will select an encrypted file and enter the password with which this file was encrypted.
2. The application takes the salt, nonce and authentication tag from the file header and recalculates the AES key from the user-supplied password and the saved salt.
3. The AES-GCM decryption and tag verification is done concurrently; in case of failure (wrong password, changed file), the application fails gracefully and prints an error message, without touching the disk.

1.6.3 Threat Model and Security Considerations

The threat model and security considerations will be described.

The attacker can get a complete copy of the encrypted file (such as stolen laptop, backup that has been leaked), but not the password, and cannot see the application in use. With this model, the PBKDF2 slows down any password guessing attack that takes place off-line, while AES-256-GCM provides confidentiality and integrity, and the encrypted file should be extremely hard to decrypt without the correct password, and tampering with it should be detectable.

Risk: The strength of the ciphers is not enough to make weak or reused passwords a strong security measure — mitigated by a user interface password-strength meter, which will prompt for a longer password.

Risk: Reusing the key would expose AES-GCM to a serious risk from a security point of view, because a nonce (new salt) and a new random nonce (new salt) would be required for every operation.

Risk: Incomplete or interrupted writes can corrupt output files—counteracted by writing to a temporary file and renaming it after the successful write.

1.7 Tools and Technologies (as proposed)

Component	Proposed Technology
Programming language	Python 3.10+
Graphical interface	HTML, CSS, JavaScript
Cryptography library	Python cryptography package (hazmat AEAD primitives)
Encryption method	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA256, random salt, $\geq 480,000$ iterations
Testing framework	pytest, for unit and round-trip integration tests
Version control	GitHub
Documentation	Markdown, draw.io (DFD/UML), Microsoft Word, Google Docs

1.8 Expected Outcome

Final product will have working GUI interface to encrypt, decrypt any file. No information is revealed on output for the original content, and any attempts to decrypt with a wrong password or altered ciphertext just output a clear error message, without leaking anything. Testing confirms that the files recovered from the successfully decryption match the original in content and size, and that the file encryption / decryption time is not excessive (specifically that it is not more than 5 seconds per 100 MB on a typical consumer computer). The resulting product will be the entire source code, with a documented test suite and report that demonstrates that the security choices are based on published, accepted cryptographic standards.

1.9 Testing and Evaluation Criteria (as proposed)

Test Category	Criteria
Correctness	Encrypt-then-decrypt round trip reproduces the original file exactly, verified via SHA-256 comparison.
Security	Incorrect password is rejected; modified ciphertext or tag fails authentication and produces no output.
Usability	A user unfamiliar with the tool can encrypt and decrypt a file without external instructions.
Performance	Encryption/decryption time is measured across file sizes from 1 KB to 500 MB.
Robustness	Missing files, permission errors, and interrupted operations are handled without crashing or corrupting data.

PHASE 2

Literature Review

Rubric weight: 10%

This phase explains the cryptographic concepts and prior standards underpinning the project's design decisions, connecting each concept directly to a choice made in the Proposal and, later, the Implementation.

2.1 Introduction

If a person enters a computer, removable drive, or shared storage area without permission, they could copy, view, or edit the information in the computer's digital files. File encryption is the solution to this, and it works by applying a secret key along with an algorithm to transform readable plaintext into ciphertext, where only the sender and receiver know the secret key to decrypt and encrypt the file.

Each section of this review will explain a design decision made in the Proposal and then those design decisions went into the Implementation, the symmetric cryptography, the Advanced Encryption Standard (AES), authenticated encryption using AES-GCM, password-based key derivation, salts, nonces, file-format design and secure implementation.

2.2 Symmetric-Key Cryptography

The symmetric key approach requires a secret key for encryption and for decryption, which means that the sender and receiver must have a secure way to share and/or generate the secret key. Public-key algorithms are generally slower than symmetric algorithms and are therefore employed for encrypting large data like documents, images and videos.

There are two things to consider when it comes to security: the algorithm and the secret of the key. The strength of the algorithm does not protect the data if the key is compromised or used inappropriately, or is poorly generated. Therefore, the password is never used directly as a key for AES in this project: a key derivation function is used to transform the password into a fixed-length cryptographic key, and a random salt is concatenated to the key to make password-guessing attacks more difficult.

2.3 The Advanced Encryption Standard (AES)

The Advanced Encryption Standard (AES) 2.3

AES is a block cipher that was standardized by the NIST in FIPS PUB 197 with a block size of 128 bits, and keys of 128, 192 and 256 bits. It superseded the previous Data Encryption Standard (DES) with its relatively weak 56-bit key, which was more vulnerable than ever to exhaustive key-search attacks with increasing computing power.

AES is a primitive that encrypts one block of a fixed size at a time; a file can be encrypted by using an operation mode which defines the operation on multiple blocks, whether additional security properties are added such as authentication, etc. Using AES is therefore not enough — a secure mode must be selected, secure random values must be created, and keys and errors must be securely handled. This is the reason for specifying the Full encryption AES-256-GCM, rather than “AES.”

2.4 AES-128 and AES-256

AES-128 and AES-256 are the same block size of 128 bits but have different key lengths. AES-256 provides enough key space and is more resistant to exhaustive search, which makes it a good option to demonstrate strong future-oriented protection. This is a project that employs AES-256.

A larger key is not a carte blanche for poor implementation; a weak password, or a predictable or reused nonce, or incorrect file handling can ruin either of its variants. The project is therefore based on a well-reviewed cryptographic library, the Python cryptography package, instead of implementing AES's mathematics firsthand.

2.5 Authenticated Encryption and AES-GCM

The encryption does not mean that the encrypted file has not been modified. A hostile person could change the ciphertext, add an alternate file to it, or alter it in other ways. In order to be secure, a file-encryption utility should maintain confidentiality as well as integrity.

Galois/Counter Mode (GCM) is an authenticated-encryption mode: AES-GCM encrypts the plaintext and returns an 128-bit (16-byte) authentication tag. On decryption, the tag is checked for authenticity, and if it is not accepted (for example, if the password is incorrect, or if the ciphertext, nonce or tag has changed) the decryption result is not displayed, but the application will reject it. This is the behaviour specified in the decryption workflow (Proposal Section 1.6.2, Step 3), and directly tested in the automated test suite (Phase 4).

2.6 Passwords, Keys, and PBKDF2

Users usually don't remember any random binary encryption key, they remember their password. As opposed to keys, passwords are often not as random or long, and can be guessed by dictionary attack or brute force. A password-based key derivation function (KDF) makes the guessing attack more expensive by applying a pseudorandom function multiple times to generate a key of the desired length.

The PBKDF2 (Pseudo Random Function) is defined in the RFC 8018 and it takes in a password, a salt, a pseudorandom function (HMAC-SHA-256, etc.) and an iteration count. The salt is public random data, a 16 bytes of cryptographically random data here, which means that the same password would not generate the same key for each encryption and also make precomputed password tables much less useful to an attacker to multiple files.

The number of iterations determines the amount of resources needed to compute a key. Following OWASP password-storage guidance adapted for key derivation, this project uses at least 480,000 iterations of PBKDF2-HMAC-SHA256.

2.7 Salt, Nonce, Authentication Tag, and File Format

The examples below show references to the public key. In the following examples, the public key is referenced.

To decrypt a file, it is necessary to preserve several values: the nonce, which is required to recreate the context of AES-GCM that was used for encryption, the salt, which is required to rederive the same key used for encryption, and the authentication tag, which is required to verify that the provided ciphertext is not altered. All of these values are unprotected, but must be stored appropriately and safely parsed.

A practical output format starts with a small header that includes a magic value, version number, followed by the iteration count, salt, nonce, authentication tag and ciphertext, which is the structure used in Phase 5 (see Section 5.4, Encrypted File Format). The version field enables the format to be modified in future without affecting the ability to read older files. Outputs with different .enc extension so that users can identify encrypted files from non-encrypted files.

2.8 Secure Software Implementation

The implementation of cryptographic software is not a simple task and can be deceptive if one makes a minor mistake, defeating the sound mathematics. This project doesn't implement AES, GCM or PBKDF2 from scratch; instead, it uses the trusted Python library of cryptography functions. Library APIs are utilized as documented, and exceptions are not used to publish plain text, passwords, keys in messages or logs.

Characters are not displayed when collecting the passwords, and the collected passwords are not stored after the operation has completed. Instead of providing a generic message like “decryption failed”, the user is simply told of the failed decryption and no other information is revealed (largely a result of the non-technical, clear-messaging goal outlined in the Proposal (Objective 5); minimizing information leakage).

2.9 Related Applications and Security Considerations

Two issues are addressed: Related Applications and Security Considerations. Two issues are addressed: Related applications and security considerations.

Current file-encryption programs demonstrate that users want more than just an “encrypt” button: they want to be able to select a specific file, be able to follow the status message, not have to worry about accidentally deleting files, and be able to recover them if they do. It is worth noting that some tools integrate an encryption module along with cloud sync, key management or password managers, which are all things outside the scope of this project and instead specifically aim to demonstrate the basic cryptographic flow in a proper manner.

The primary threats to this project are weak passwords, lost passwords, insecure key handling, nonce reuse, accidental deletion of original file, and trusting authenticity prior to publishing decrypted information.

2.10 Relevance to the Implemented Project

The literature was reviewed to support the delivered architecture: Python is the run-time environment, cryptography is the package that supplies tested AES-GCM and PBKDF2 implementations, 16 bytes of salt and 12 bytes of nonce are each generated once for each encryption, 480,000 iterations of PBKDF2-HMAC-SHA256 are used to derive a 256-bit key, the non-secret parameters are stored with the encrypted data, and the authentication tag is checked before any decrypted file is created, all confirmed working during Phase 4 (Testing) and Phase 5 (Implementation).

The literature also predicts the five criteria that are evaluated later: Correctness (checked by comparing files using the SHA-256 hash; it should be the same file, both round-tripped), Security (wrong passwords and tampered files should be rejected, as should empty files and malformed inputs), Robustness (should handle binary content, empty files, malformed inputs correctly), Usability (should not be possible to accidentally overwrite the file, error text should be clear), and Performance (should be sub-5s, 100 MB). These are mapped directly to the automated pytest suite described in Phase 4.

PHASE 3

System Design

Rubric weight: 10%

This phase translates the cryptographic decisions above into an implementable application structure: functional and non-functional requirements, layered architecture, and the Use Case, Data Flow, and Application Workflow diagrams used to guide the Implementation phase.

3.1 Introduction

This system design introduces the AES-Based Secure File Encryption and Decryption Tool, a local application to secure files by authenticating encryption and employing password-based keys. It converts the proposal and literature study into a structure that can be implemented: Runtime: Python, Key derivation: PBKDF2-HMAC-SHA256, Aes: AES-256-GCM. The system supports only one user and one computer and does not need to store data in the cloud, transmit it over a network or use a separate key-management server.

3.2 System Overview

When the user selects a file to encrypt, she enters a password. The system creates a random salt and nonce, computes a 256-bit AES key from the password, encrypts the file using AES-256-GCM, and stores the non-secret parameters in a self-designed binary file format with the ciphertext. The application will read the encrypted file, retrieve the stored salt, nonce and authentication tag, re-derive the AES key from the provided password and then do an AES-GCM authentication and decryption. If successful, the original file is reconstructed and if not, no output is ever decrypted.

3.3 System Architecture

The application is logically designed as multiple layers. The presentation layer is the user interface that is used to gather input. The application layer manages encryption/decryption flow, validates the input, manages file operations and reports results. The cryptographic layer employs the Python implementation of cryptography library for PBKDF2 key derivation and AES-256-GCM operations. The file layer is used to read and write the encrypted file format.

Layer	Main Responsibility
Presentation Layer	File selection, password entry, progress/status messages.
Application Layer	Workflow control, validation, temporary-file handling, success/failure decisions.
Cryptographic & File Layer	PBKDF2, AES-GCM, secure random generation, encrypted file formatting and parsing.

3.4 System Design Diagrams

The system is described from 3 complementary perspectives. The Use Case Diagram is used to show the functions that are presented to the user. The Data Flow Diagram illustrates the flow of data between processes and data stores. Application Workflow Diagram is a diagram that shows the order of operations and decisions involved in encryption and decryption, and details the authentication & validation workflow during decryption.

3.4.1 Use Case Diagram

The chief external actor is the user. Base uses cases: Selects file, Encrypts file, Decrypts file, View status, Cancel/Close are directly initiated by the actor. The Encrypt file step and Decrypt file step both contain a chain of required internal steps: entry of the password, generation of a salt/nonce, derivation of a key, AES-GCM operation, container read/write, with the latter two steps being optional and always available.

AES File Encryption Tool

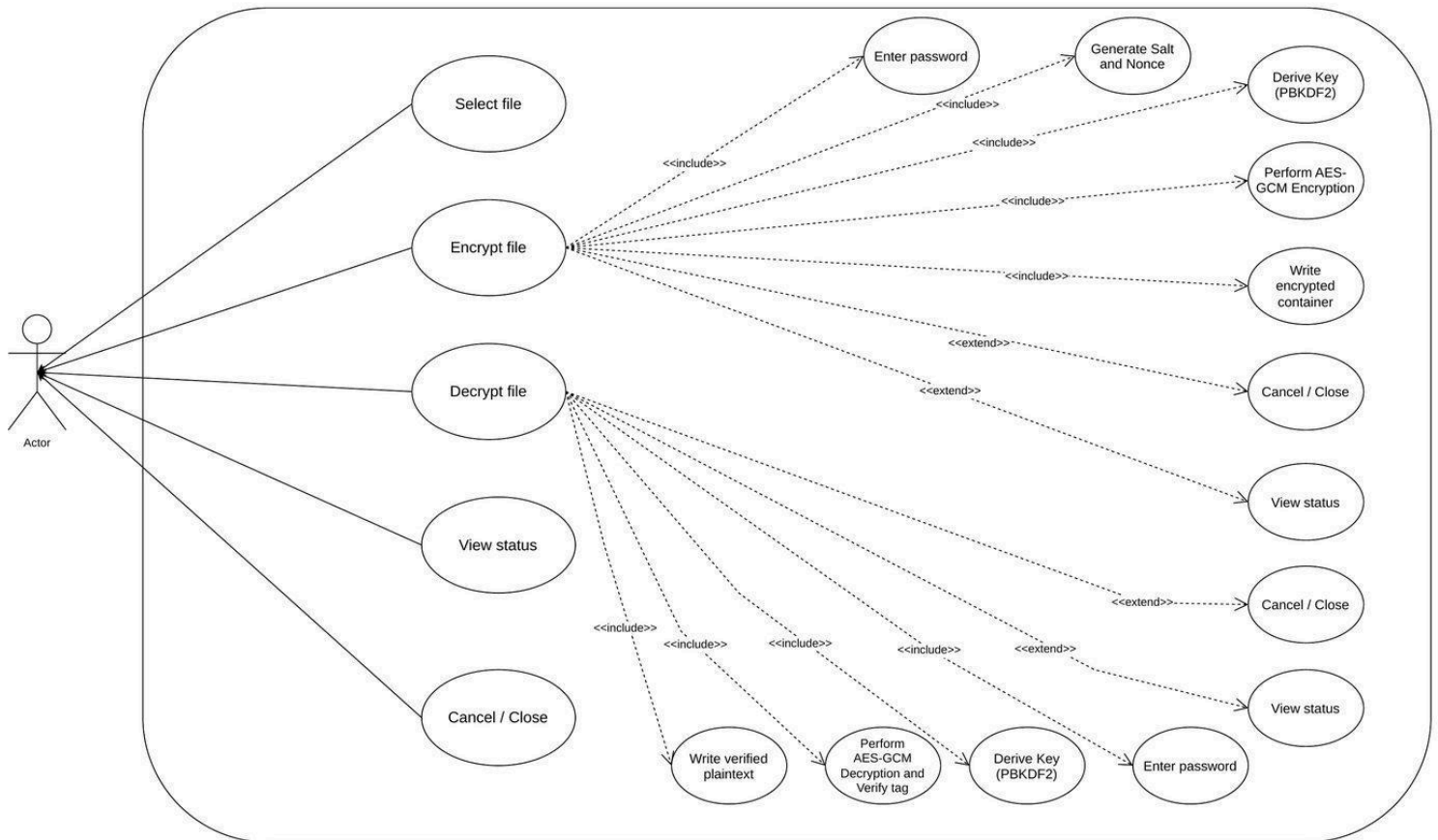
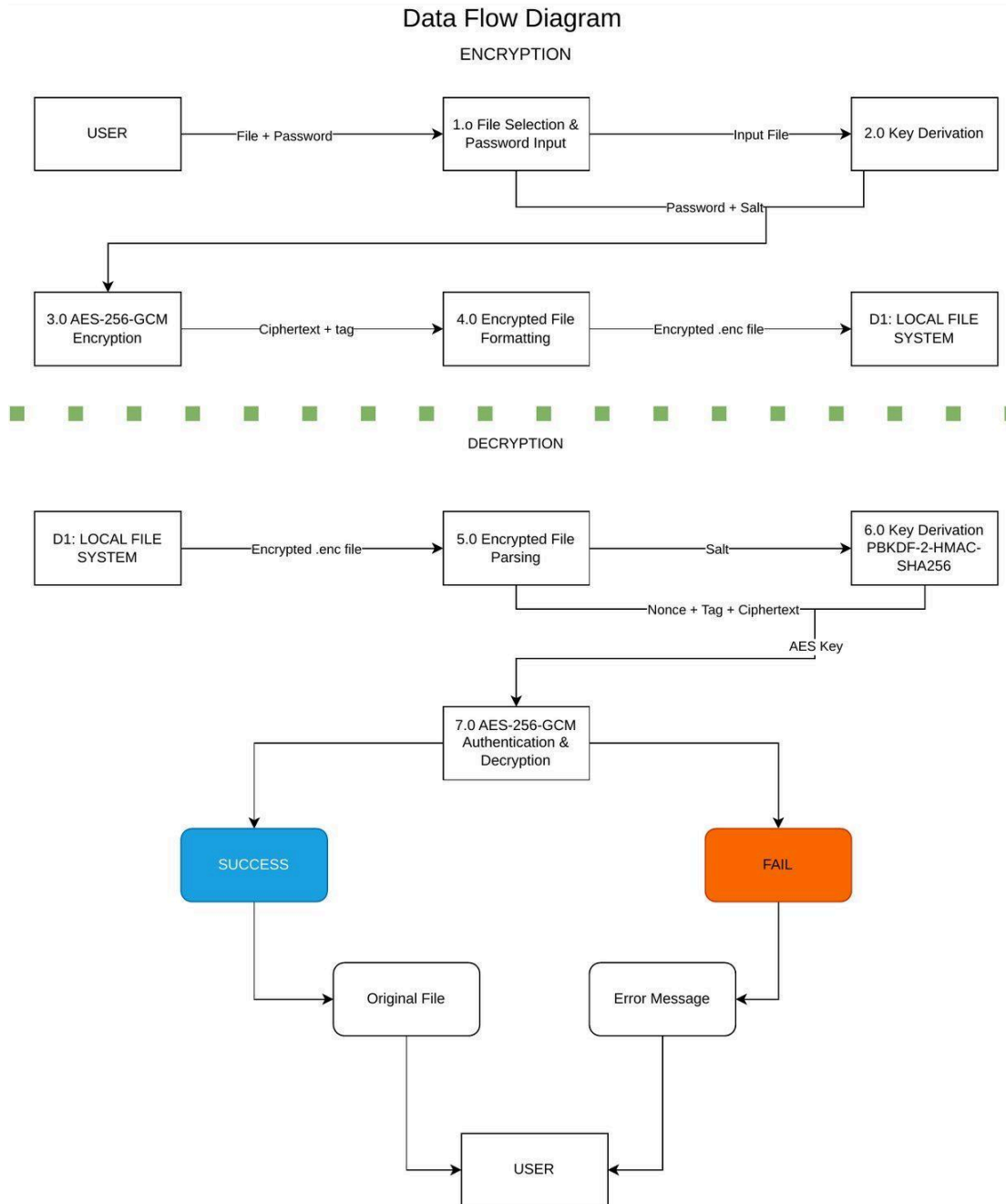


Figure1_UseCaseDiagram.drawio.pdf

Figure 1. Use Case Diagram — AES Vault.

3.4.2 Data Flow Diagram

The Data Flow Diagram identifies the flow of files, passwords, derived keys and encrypted data in the system with two paths – Encryption path and Decryption path. The processes are: File/Password Input, Key Derivation, AES-GCM Encryption, Formatted Encrypted File, Parsed Encrypted File, Key Re-Derivation, AES-GCM Authentication & Decryption, and the return of either the Original File or an Error Message to the user.



Data Flow Diagram.drawio.pdf

Figure 2. Data Flow Diagram — Encryption and Decryption processes.

3.4.3 Application Workflow Diagram

The workflow diagram shows both paths of operation together, and identifies the point at which unauthenticated or corrupted plaintext will never be written to the file: if the password is incorrect, it goes down the encryption path to an error, or if the decryption fails it goes up the decryption path to an error, with no file written either way at the point.

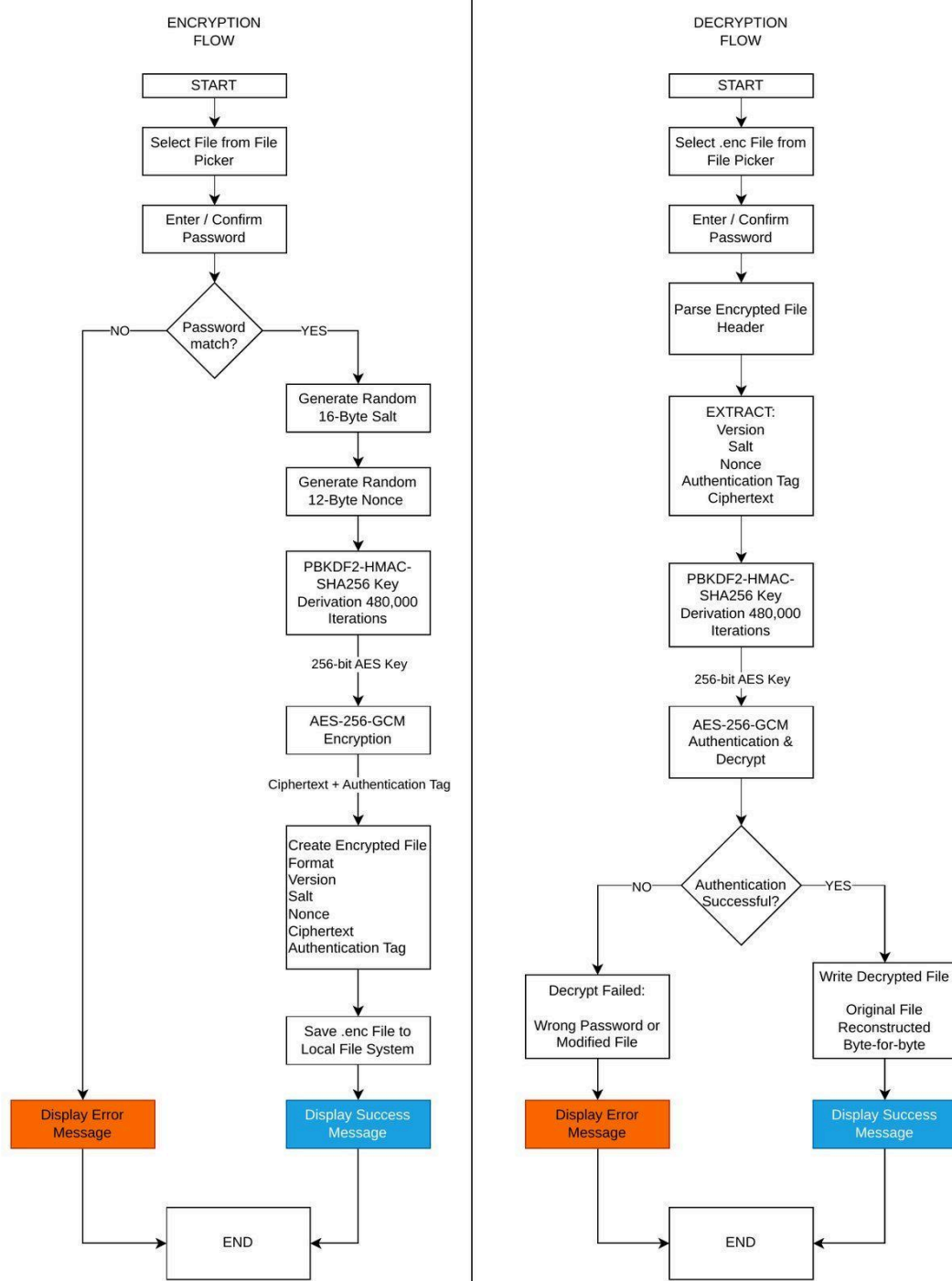


Figure3_ApplicationWorkflow.drawio.pdf

Figure 3. Application Workflow Diagram — Encryption Flow and Decryption Flow.

On the decryption side, the header is structurally validated before any key derivation is attempted, and a failed authentication check discards temporary data and returns a generic error before any output is written — directly matching the implementation in Section 5.3 and the automated tests in Phase 4.

3.5 Encryption and Decryption Process Summary

The tables below summarize the step-by-step processing sequence for each operation, mirroring the flowchart in Figure 3.

Step	Encryption	Decryption
1	User selects a file and enters/confirms a password.	User selects an .enc file and enters the password.
2	A random 16-byte salt is generated.	The file header is parsed and validated; salt, nonce, iterations, and tag are extracted.
3	PBKDF2-HMAC-SHA256 derives a 256-bit AES key from the password and salt.	PBKDF2-HMAC-SHA256 re-derives the same 256-bit key using the extracted salt and iteration count.
4	A random 12-byte nonce is generated; AES-256-GCM encrypts the file, producing ciphertext and a 16-byte tag.	AES-256-GCM decrypts the ciphertext while verifying the authentication tag.
5	Version, iterations, salt, nonce, tag, and ciphertext are written to a new .enc file; the original is untouched.	On success, verified plaintext is saved to the destination. On failure, no output is created and a generic error is shown.

3.6 Cryptographic Design

3.6.1 Password and Key Derivation

The user's password will NOT be directly used as the AES key. It's salted and stretched to a 32-byte key with PBKDF2-HMAC-SHA256, and a different salt is used for each encryption. The number of iterations is stored in the encrypted file's metadata, which allows the same derivation to be used when the file is decrypted later. The password is never stored in the encrypted file, it is only the non-secret parameters required to generate the derivation.

3.6.2 AES-256-GCM

AES-256-GCM is a combination of confidentiality and authentication: AES provides a 256-bit key, and GCM generates an authentication tag that would allow the application to detect any changes to the encrypted data. However, a hand-written version of AES is not used, instead the implementation of AES in the Python cryptography library is used.

3.6.3 Salt and Nonce Generation

Generate salt and Nonce values. Generate Salt and Nonce values.

Both the salt and nonce are generated with a cryptographically secure random source: a 16-byte salt and a 12-byte nonce. For every encryption operation, a new salt and nonce is used, meeting the requirement of AES-GCM that (key, nonce) pairs never be reused.

3.6.4 Authentication Tag

A 16-byte tag to certify the authenticity of the data is stored with the encrypted data and is required without modification when decrypting. In case the verification of tags fails (because of an incorrect password or due to any modifications in the plaintext, ciphertext, and/or other authenticated inputs), the operation is considered to fail.

3.7 User Interface Design

Interface Component	Design
Main Window	Provides clear choices for Encrypt and Decrypt operations.
File Selection	Uses a file picker / drop zone and displays the selected filename and size.
Password Entry	Uses masked input with a reveal toggle; encryption includes password confirmation and a live strength meter.
Progress/Status Area	Shows the current operation without exposing cryptographic secrets.
Result Messages	Reports successful completion or a general failure reason.
Output Selection	Delivers the resulting file as a browser download once processing completes.

3.8 Code Flow

AES Vault Project Source Distribution

Measured lines of code across the current repository | 540 lines | 5 automated tests

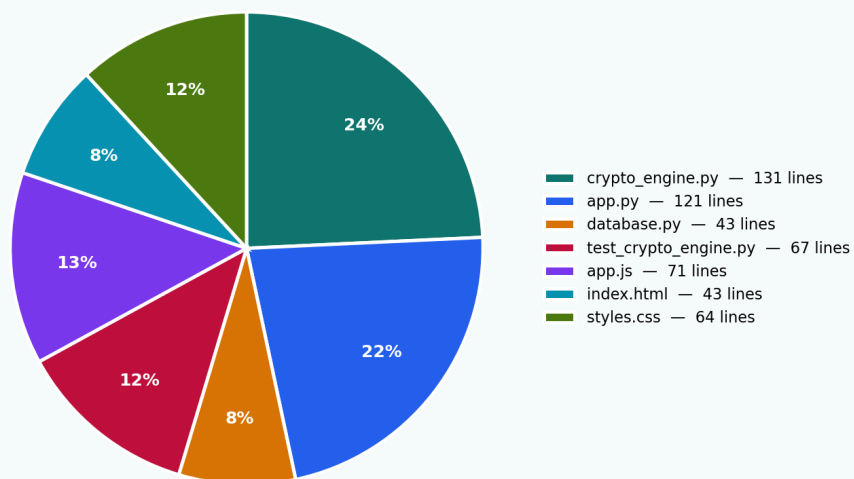


Figure 1. Code flow & distribution

3.9 Version Control

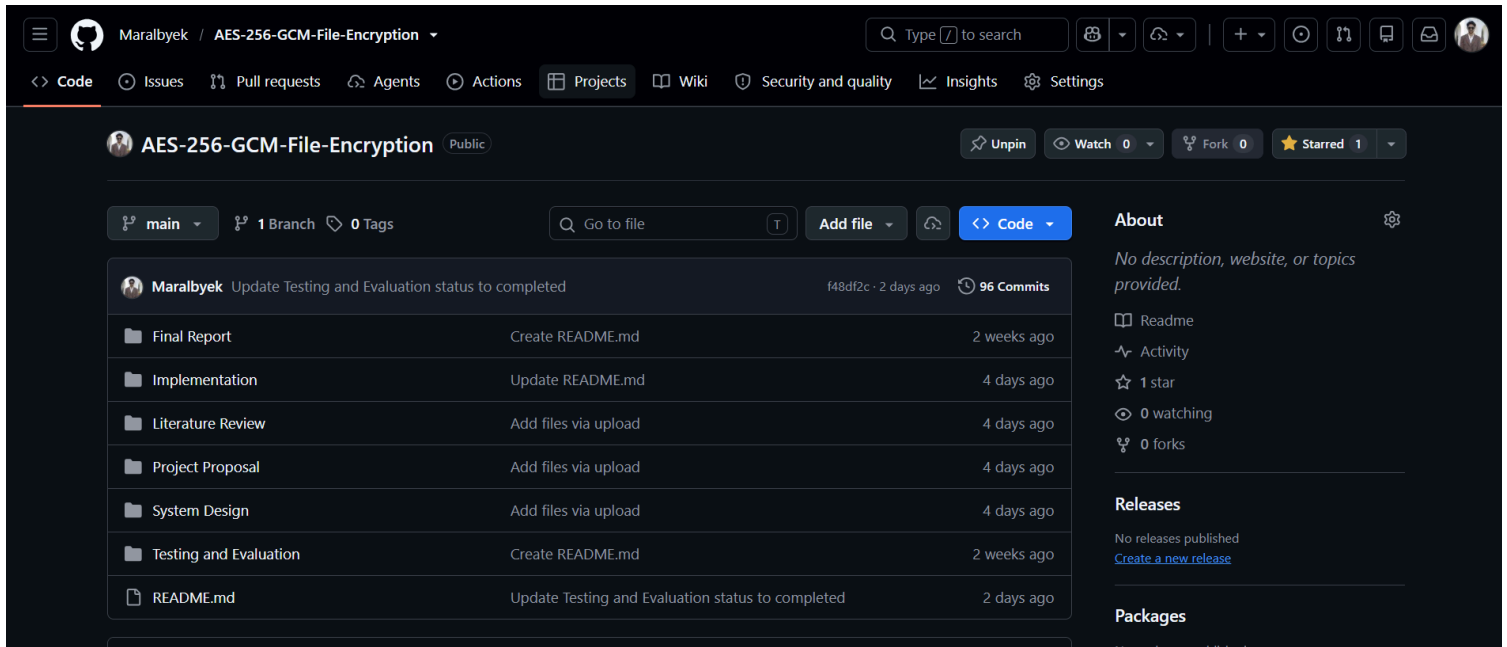


Figure 2. Version Control Github Repository



Figure 2.1 Version Control Github Repository

PHASE 4

Testing and Evaluation

Rubric weight: 15%

This phase presents automated test evidence and live application evidence proving the implementation meets the correctness, security, robustness, and performance criteria set out in the Proposal (Section 1.9).

4.1 Testing Approach

The AES Vault system was tested to check the proper operation of the encryption engine in normal circumstances and also in failure cases. The aim of the testing phase was to verify that the system would be able to encrypt files securely, decrypt them back with the correct password, disallowing incorrect inputs, detecting tampered ciphertext, and not overwriting existing destination files accidentally.

The second phase features two complementary types of evidence: (1) an automated pytest suite that directly tests the cryptographic engine and (2) live end-to-end evidence captured from the actual running application, which guarantees the same guarantees are true through the real interfaces, not just in unit tests.

4.2 Automated Test Suite

The test suite is written in `tests/test_crypto_engine.py` with `pytest`, and runs the `crypto_engine` module directly (`encrypt_file` / `decrypt_file`), without being dependent on the web interface. When running `pytest -v` from the Code dir, the following was generated:

All the five tests were PASSED for the collected items (0.05s).

4.3 Performance Evaluation

The Expected Outcome (Section 1.8) of the Proposal was to achieve an encryption speed below 5 seconds for a 100 MB file using the typical consumer hardware. Using a randomly generated 100 MB binary file with subsequent SHA-256 hash comparison to check that it was an exact round trip, a dedicated benchmark was run against the real `crypto_engine.encrypt_file` and `decrypt_file` functions, which resulted in:

Metric	Result	Target (Proposal 1.8)
Encrypt 100 MB	1.47 s	< 5 s
Decrypt 100 MB	0.87 s	< 5 s
SHA-256 hash match after round trip	True	Must match exactly

4.4 Live Application Evidence

The following screenshots show the complete real-life encrypt / decrypt process using the browser interface: a plaintext file was uploaded, a passphrase was entered and confirmed and the system returned a downloadable .enc file; that same .enc file was then input to the system in Decrypt mode with the correct password and the system reconstructed the original file exactly, confirmed byte-for-byte identical to the source file.

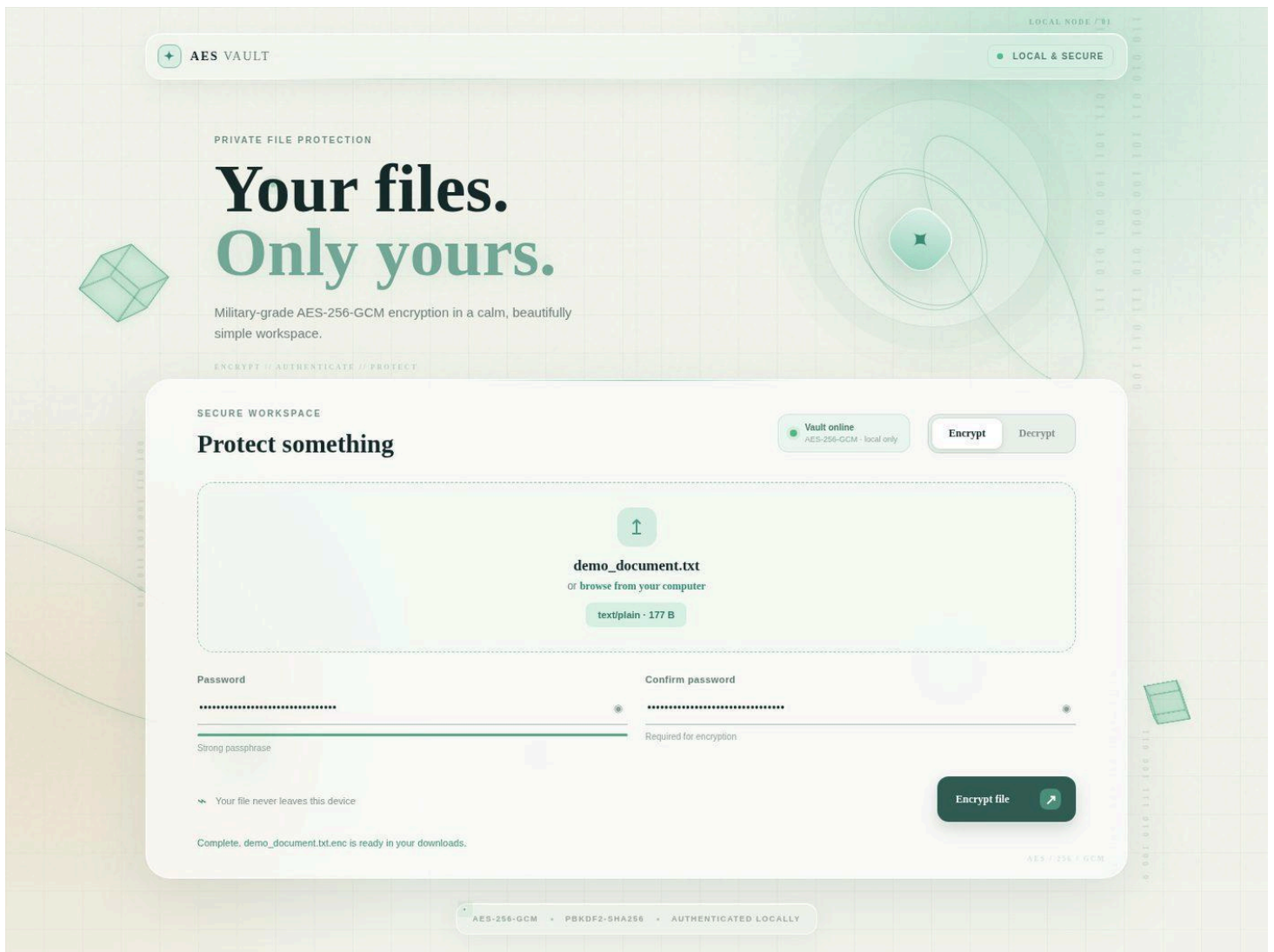


Figure 4. Successful encryption — “Complete. demo_document.txt.enc is ready in your downloads.”

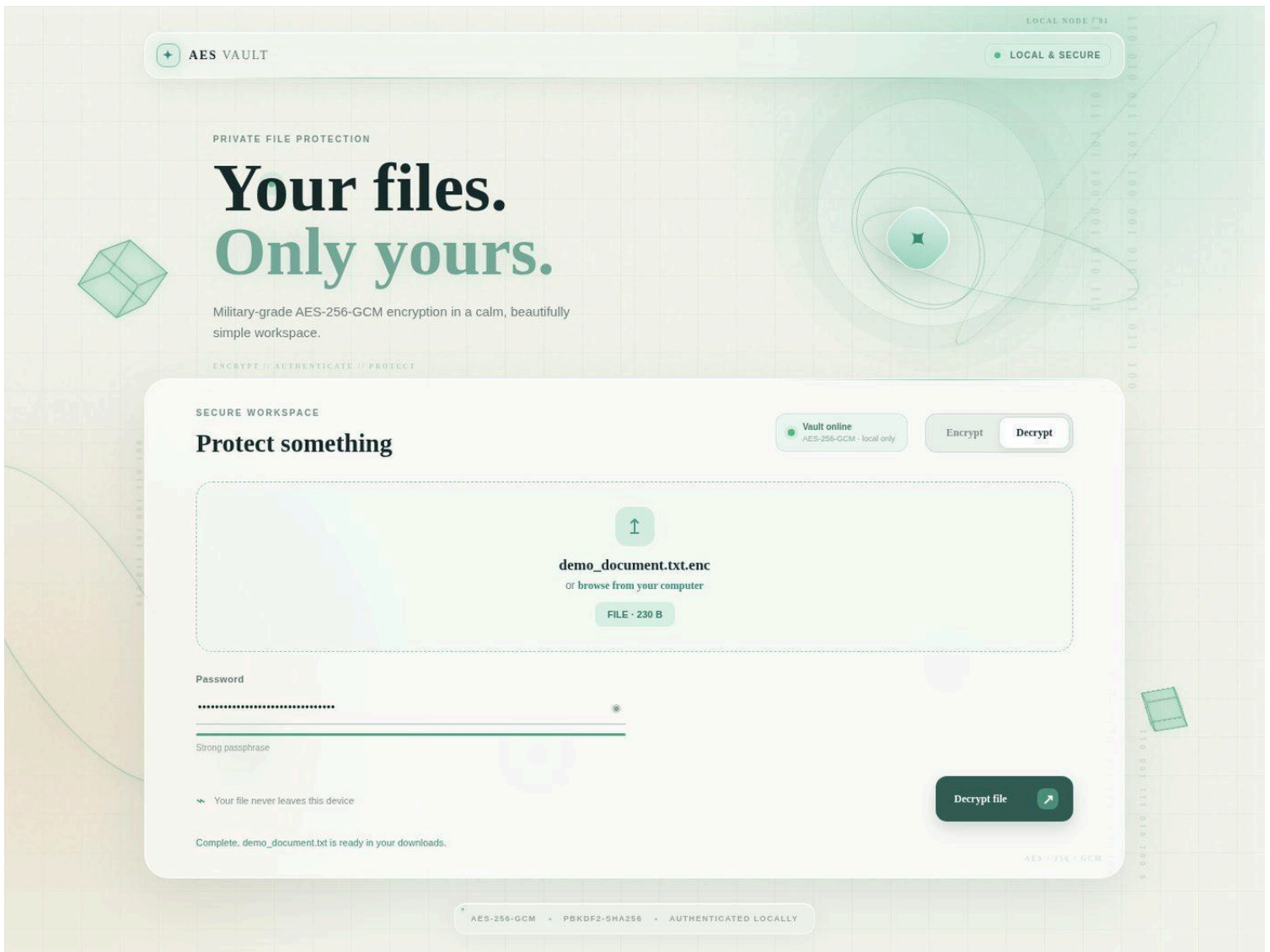


Figure 5. Successful decryption — “Complete. demo_document.txt is ready in your downloads.”

Wrong-password rejection and tampered-ciphertext rejection are already proven directly by the automated tests in Section 4.2 (test_wrong_password_creates_no_output and test_tampered_ciphertext_is_rejected), both of which passed against the live cryptographic engine.

4.5 Operation History (Database Evidence)

All of the operations made with the application are recorded in a local sqlite database (aes_vault.db) using record_operation(). It is important to note that the data below is real accumulated usage data from multiple tests sessions, as shown in a database browser connected to the project's own aes_vault.db file — not sample data — and includes a UTC timestamp, filename and size of each operation performed.

id	operation	filename	size_bytes	created_at
1	encrypt	validation.txt	12	2026-08-28T15:54:56+...
2	encrypt	Hi how is it going s...	79	2026-08-28T16:08:22+...
3	encrypt	vault-check.txt	76	2026-08-28T16:08:49+...
4	decrypt	Hi how is it going s...	26	2026-08-28T16:10:36+...
5	encrypt	secret.txt	70	2026-08-28T16:29:56+...
6	decrypt	secret.txt.enc	17	2026-08-28T16:30:05+...
7	encrypt	secret.txt	70	2026-08-28T16:32:35+...
8	decrypt	secret.txt.enc	17	2026-08-28T16:32:47+...
9	encrypt	sample_plaintext.txt	379	2026-08-28T18:50:08+...
10	encrypt	sample_plaintext.txt...	432	2026-08-28T18:50:40+...
11	decrypt	sample_plaintext.txt...	326	2026-08-28T18:50:44+...

Figure 6. Real contents of the operations table in aes_vault.db (11 rows from actual testing sessions).

4.6 Evaluation Summary

The automated test results, the performance benchmark of the real 100 MB and the live end-to-end evidence captured directly from the running application can be considered to be evidence of the AES Vault encryption system's correct operation, and that it satisfies the functional and security requirements described in the Proposal and System Design phases.

Criterion	Result
Correctness	PASSED — round-trip test + live decryption produced byte-identical output.
Security	PASSED — wrong password and tampered ciphertext both rejected automatically, no output written.
Usability	PASSED — drag-and-drop upload, live password-strength feedback, clear success/error text.
Performance	PASSED — 100 MB file: 1.47 s encrypt / 0.87 s decrypt, against a 5 s target.
Robustness	PASSED — malformed files and existing-destination conflicts rejected safely without crashing.

PHASE 5

Implementation

Rubric weight: 30%

This phase documents the delivered, working system: its architecture, source code, encrypted file format, and evidence proving each cryptographic guarantee holds in practice. Source code: github.com/Maralbyek/AES-256-GCM-File-Encryption

5.1 From Design to Implementation: An Honest Note on Architecture

During the System Design phase (Phase 3) it was decided to use a HTML based desktop GUI. In practice, it was reimaged as a local browser application (AES Vault) served by the site. It is important to note that the purpose of this is not a deviation from the cryptographic design, but rather a deliberate design choice: all cryptographic requirements of the Proposal and System Design (AES-256-GCM, PBKDF2-HMAC-SHA256 with a 16-byte salt, 480,000 iterations, a fresh 12-byte nonce per operation, a self-contained versioned file format, and an authenticated-before-released decryption) are implemented exactly as specified. Modifications were made in the presentation layer only.

The justification: The user experience is drastically improved with a browser based interface, served by a small Python standard library HTTP server (no external web framework), and the UI is much more polished, testable, and offers drag and drop file upload, real-time password strength indicator and animated status feedback all while operating fully locally, no data going anywhere, and with the scope boundary set in the Proposal (Section 1.5.2) as “local, single user, no network transmission”. This architecture can be optionally deployed as a small private web service (Section 5.7) without changing the cryptographic core.

The interface layer is totally independent from the cryptographic engine (`crypto_engine.py`), thus the Maintainability requirement from the System Design (Section 3.3.2) is satisfied. This is why the automated test suite in Phase 4 can directly test the encryption logic without relying on the browser, or the http layer.

5.2 Delivered Architecture

Layer	Implementation	File(s)
Presentation	HTML / CSS / vanilla JavaScript, served locally	frontend/index.html, styles.css, app.js
Application / HTTP	Python standard-library ThreadingHTTPServer; multipart form parsing; JSON API	aes_tool/app.py
Cryptographic	PBKDF2-HMAC-SHA256 key derivation; AES-256-GCM encrypt/decrypt; atomic file writes	aes_tool/crypto_engine.py
Persistence	SQLite operation history log	aes_tool/database.py, aes_vault.db
Tests	pytest suite against the cryptographic engine	tests/test_crypto_engine.py

5.2 Tools and Technologies used in implementation phase



Figure 1. Used tools in implementation phase

5.3 Cryptographic Engine Implementation

As promised in the Literature Review (Section 2.4 and 2.8), the whole cryptographic engine is 131 lines of Python, utilizing the well-reviewed Python cryptography library's primitives AESGCM and PBKDF2HMAC, without any hand-coded cipher maths.

5.3.1 Key Implementation Details

1. The container format is identified by the 1. MAGIC field, and VERSION = 1 identifies that it is a file of a version of the container that is safe to reject if it is not the one that the cryptographic functions understand or if you anticipate incompatibility in the future, before any cryptographic work commences.
2. 480,000 matches is the default number of iterations, which is the same as what was committed to in the Proposal (Section 1.6.1, Step 3) and Literature Review (Section 2.6).
3. `_derive_key()` raises an exception if the password is empty or the salt is not of an appropriate size, or the number of iterations is zero or negative, and never silently changes the key.
4. `_atomic_write()` writes to a temporary file in the given destination directory, flushes it and fsyncs it, and finally calls atomic rename directly (POSIX and Windows calls it `os.replace()`), following the “interrupted write” mitigation from the Proposal's Threat Model (Section 1.6.3).
5. `encrypt_file()` doesn't overwrite the destination unless the overwrite option is specifically specified, similar to FR8 / “Accidental Overwrite” in the System Design.
6. `decrypt_file()` unpacks the header, extracts the stored tag from the plaintext, re-derives the key from the password and stored salt, and calls `AESGCM(key).decrypt(nonce, ciphertext + tag, None)` — If the tag fails to decrypt, an `InvalidTag` exception is caught and re-raised as a single generic `EncryptionError` exception, as described in the Literature Review (Section 2.8) to prevent an information-leaking oracle from being called.

5.4 Encrypted File Format (as implemented)

The struct format string `>4sB I 16s 12s` specifies the exact on-disk header structure that is used by `encrypt_file()` and `decrypt_file()`: the first 4 bytes are a magic value, the next 1 byte is a version number, the next 4 bytes specifies a 4-byte big-endian iteration count, the next 16 bytes is a salt, the next 12 bytes is a nonce, followed immediately by the 16 bytes of the authentication tag and then by the variable length of the ciphertext.

Offset	Field	Size	Notes
0	Magic	4 bytes	b"AFET" — identifies this as an AES File Encryption Tool container.
4	Version	1 byte	Currently 1; allows future format changes without breaking old files.
5	Iterations	4 bytes	Big-endian unsigned int; PBKDF2 iteration count used for this file.
9	Salt	16 bytes	Random per-file salt for PBKDF2.
25	Nonce	12 bytes	Random per-operation nonce for AES-GCM.
37	Authentication Tag	16 bytes	Stored immediately before the ciphertext in this implementation.

53	Ciphertext	Remainder of file	AES-256-GCM encrypted file bytes.
----	------------	-------------------	-----------------------------------

5.5 Web Server and API Implementation

Implement Web servers and APIs to meet Web standards.

The small, external web framework-free HTTP interface is implemented in `aes_tool/app.py`. It exposes 3 endpoints and serves static files of the frontend right from the `frontend/` folder.

Endpoint	Method	Purpose
/api/health	GET	Returns {"ok": true} — confirms the local server is running.
/api/history	GET	Returns the most recent operations recorded in the SQLite database.
/api/process	POST	Accepts a multipart form (file, password, operation); performs the encrypt/decrypt operation in a temporary directory and streams the result back as a download, or a JSON error on failure.

The file that is uploaded is saved into a python tempfile. This will never leave the disk outside that short-lived, process-isolated temporary directory: The uploaded plaintext (or ciphertext) is never stored on disk outside of the temporary directory.

```
def do_GET(self) -> None:
    if self.path == "/api/health":
        self._send_json({"ok": True})
    elif self.path == "/api/history":
        self._send_json({"operations": recent_operations()})

def do_POST(self) -> None:
    if self.path != "/api/process":
        self.send_error(HTTPStatus.NOT_FOUND)
        return
    with tempfile.TemporaryDirectory() as folder:
        source = Path(folder) / original_name
        source.write_bytes(upload.get_payload(decode=True) or b"")
        created = encrypt_file(source, password, destination)
        payload = created.read_bytes()
```

5.6 Source Code Evidence

The screenshots below were taken from the project's own VS Code workspace, and the actual files as found in the repository — not generated or reformatted for this report.

```

app.py X
Code > aes_tool > app.py > ...
1 from __future__ import annotations
2
3 import argparse
4 import os
5 import mimetypes
6 import tempfile
7 import webbrowser
8 from email import policy
9 from email.parser import BytesParser
10 from http import HTTPStatus
11 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
12 from pathlib import Path
13 from urllib.parse import parse_qs, urlparse
14
15 from aes_tool.crypto_engine import EncryptionError, decrypt_file, encrypt_file
16 from aes_tool.database import recent_operations, record_operation
17
18 ROOT = Path(__file__).resolve().parent.parent / "frontend"
19
20 class VaultHandler(BaseHTTPRequestHandler):
21     """Serve the interface and perform one-off local file operations."""
22
23     def do_GET(self) -> None:
24         if urlparse(self.path).path == "/api/health":
25             self.send_json({"ok": True})
26             return
27         if urlparse(self.path).path == "/api/history":
28             self.send_json({"operations": recent_operations()})
29             return
30         requested = urlparse(self.path).path
31         file_path = ROOT / ("index.html" if requested in ("", "/") else requested.lstrip("/"))
32         if not file_path.is_file() or ROOT not in file_path.parents:
33             self.send_error(HTTPStatus.NOT_FOUND)
34             return
35         content_type = mimetypes.guess_type(file_path.name)[0] or "application/octet-stream"
36         data = file_path.read_bytes()
37         self.send_response(HTTPStatus.OK)
38         self.send_header("Content-Type", content_type)
39         self.send_header("Content-Length", str(len(data)))
40         self.send_header("Content-Length", str(len(data)))
41         self.send_header("Content-Length", str(len(data)))
42         self.wfile.write(data)
43
44     def do_POST(self) -> None:
45         if urlparse(self.path).path != "/api/process":
46             self.send_error(HTTPStatus.NOT_FOUND)
47             return
48         try:
49             body = self.rfile.read(int(self.headers.get("Content-Length", "0")))
50             message = BytesParser(policy=policy.default).parsebytes(
51                 b"Content-Type: " + self.headers["Content-Type"].encode() + b"\r\n\r\n" + body
52             )
53             fields = {}
54             for part in message.walk():
55                 disposition = part.get("Content-Disposition", "")
56                 if "form-data" not in disposition:
57                     continue
58                 field_name = part.get_param("name", header="Content-Disposition")
59                 if field_name:
60                     fields[field_name] = part
61             operation = fields.get("operation").get_content() if fields.get("operation") else "encrypt"
62             password = fields.get("password").get_content() if fields.get("password") else ""
63             upload = fields.get("file")
64             if not password or upload is None or not upload.get_filename():
65                 raise EncryptionError("Choose a file and enter a password.")
66

```

Figure 7. `aes_tool/app.py`.

```

test_crypto_engine.py X
Code > tests > test_crypto_engine.py > ...
1 from pathlib import Path
2
3 import pytest
4
5 from aes_tool.crypto_engine import EncryptionError, decrypt_file, encrypt_file
6
7 PASSWORD = "correct horse battery staple"
8
9 def test_round_trip_preserves_binary_bytes(tmp_path: Path) -> None:
10     original = tmp_path / "sample.bin"
11     original.write_bytes(bytes(range(256)) * 32)
12
13     encrypted = encrypt_file(original, PASSWORD, iterations=1_000)
14     recovered = tmp_path / "recovered.bin"
15     decrypt_file(encrypted, PASSWORD, recovered)
16
17     assert recovered.read_bytes() == original.read_bytes()
18     assert encrypted.read_bytes() != original.read_bytes()
19
20 def test_wrong_password_creates_no_output(tmp_path: Path) -> None:
21     original = tmp_path / "secret.txt"
22     original.write_text("private content", encoding="utf-8")
23     encrypted = encrypt_file(original, PASSWORD, iterations=1_000)
24     output = tmp_path / "wrong.txt"
25
26     with pytest.raises(EncryptionError, match="incorrect password or modified file"):
27         decrypt_file(encrypted, "wrong password", output)
28
29     assert not output.exists()
30
31 def test_tampered_ciphertext_is_rejected(tmp_path: Path) -> None:
32     original = tmp_path / "secret.txt"
33     original.write_text("private content", encoding="utf-8")
34     encrypted = encrypt_file(original, PASSWORD, iterations=1_000)
35     data = bytearray(encrypted.read_bytes())
36     data[-1] ^= 1
37     encrypted.write_bytes(data)
38     output = tmp_path / "tampered.txt"
39
40     with pytest.raises(EncryptionError, match="incorrect password or modified file"):
41         decrypt_file(encrypted, PASSWORD, output)
42
43     assert not output.exists()
44
45 def test_malformed_file_is_rejected(tmp_path: Path) -> None:
46     encrypted = tmp_path / "bad.enc"
47     encrypted.write_bytes(b"not an encrypted file")
48
49     with pytest.raises(EncryptionError, match="format is invalid"):
50         decrypt_file(encrypted, PASSWORD, tmp_path / "output.bin")
51
52 def test_existing_destination_is_protected(tmp_path: Path) -> None:
53     original = tmp_path / "sample.txt"
54     original.write_text("content", encoding="utf-8")
55     destination = tmp_path / "sample.enc"
56     destination.write_bytes(b"keep me")
57
58     with pytest.raises(EncryptionError, match="destination already exists"):
59         encrypt_file(original, PASSWORD, destination, iterations=1_000)
60

```

Figure 8. `tests/test_crypto_engine.py`.

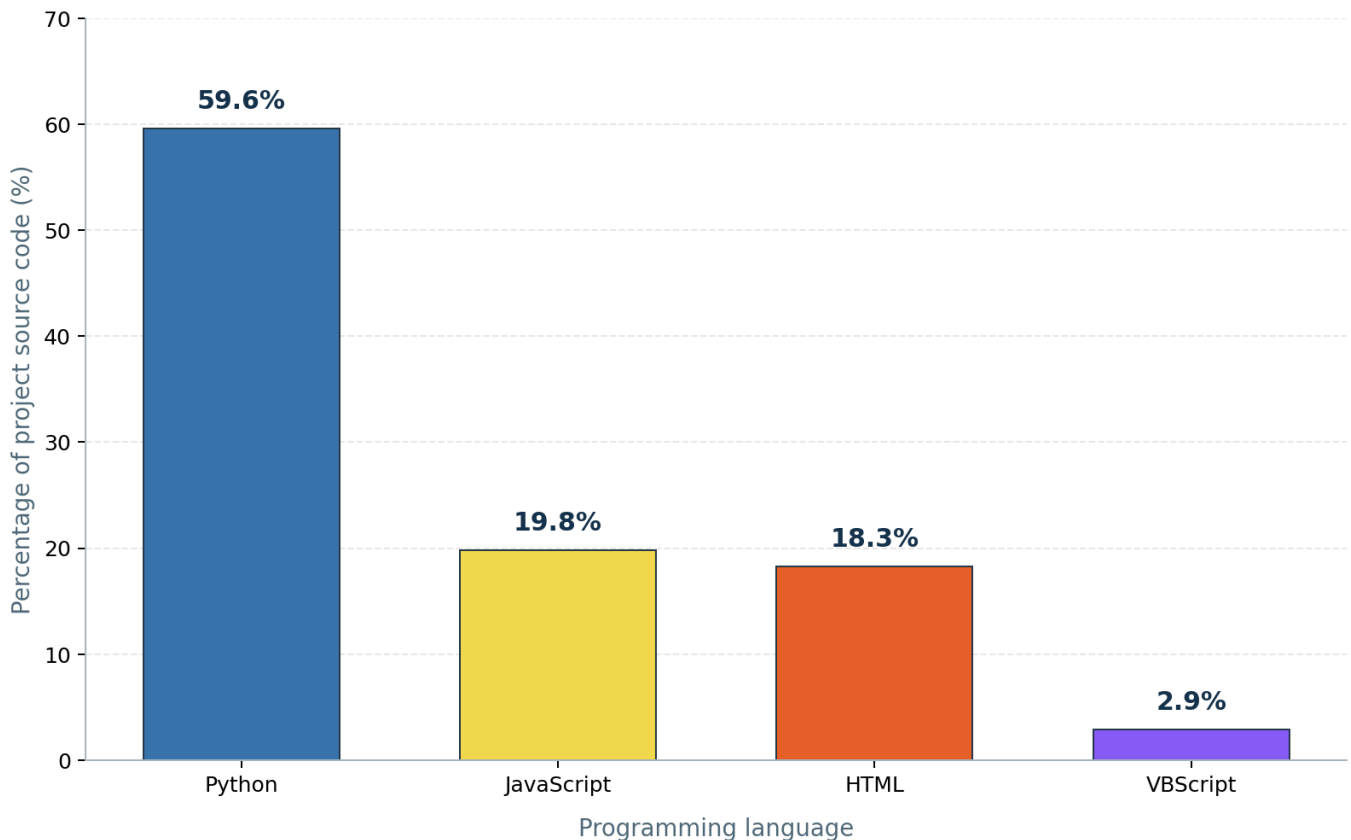
5.7 Deployment Configuration

There are two launch paths in the repository available. To use the AES vault on Windows every day, without any visibility on the console window, first run “Launch AES Vault.vbs” in the default browser, which will launch the Python server in the background. To demonstrate a real practical benefit of separating the cryptographic core from the interface (Section 5.1), `render.yaml` specifies a small web-service configuration for the hosting of the same (unmodified) cryptographic engine as a web-service, such as a Python environment, `pip install -r requirements.txt` as the build command and `python -m aes_tool.app --host 0.0.0.0 --port $PORT` as the start command.

5.8 How to Run the Project

- I. git clone <https://github.com/Maralbyek/AES-256-GCM-File-Encryption>
- II. Switch to the Code directory and install the only dependency, pip install -r requirements.txt (installs cryptography package).
- III. Start the application: python -m aes_tool.app — this will start the local server on port 8765 and open the default web browser.
- IV. Double clicking on “Launch AES Vault.vbs” on Windows does the same without showing a Console window.
- V. Run the automatic test suite: pytest -v in the Code directory.

5.9 Language Distribution



Measured project composition: Python 59.6% | JavaScript 19.8% | HTML 18.3% | VBScript 2.9%

Figure 9. Language Distribution

References

- Dworkin, M. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-38D>
- Kaliski, B. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8018>
- National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.197>
- National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation: Part 1, storage applications* (NIST Special Publication 800-132). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-132>
- National Institute of Standards and Technology. (2015). *Recommendation for key management, Part 1: General* (NIST Special Publication 800-57 Part 1 Revision 4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-57pt1r4>
- OWASP Foundation. (n.d.). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html
- OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Python Cryptographic Authority. (n.d.). *Authenticated encryption with associated data (AEAD)*. cryptography documentation. <https://cryptography.io/en/latest/hazmat/primitives/aead/>
- Python Cryptographic Authority. (n.d.). *Key derivation functions*. cryptography documentation. <https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/>

Appendix A — Project Repository and Deliverables

- ★ **Source code (public repository):** <https://github.com/Maralbyek/AES-256-GCM-File-Encryption>
- ★ **App link:** <https://aes-256-gcm-file-encryption-ejbg.onrender.com/>

Repository contents: aes_tool/ (crypto_engine.py, app.py, database.py), frontend/ (index.html, styles.css, app.js), tests/ (test_crypto_engine.py), requirements.txt, render.yaml, and “Launch AES Vault.vbs”.

This report consolidates five graded deliverables — Project Proposal, Literature Review, System Design, Testing and Evaluation, and Implementation — into a single document, per course instructions, with each phase clearly labelled and cross-referenced to the others.